Day 23 – Insert Node in Linked List

🚀 Linked List me Insert Node (Complete Guide)

🧠 Basic Concept

Linked List me insertion ka matlab hai ek naya node banana aur usko correct position par connect karna.

👉 Har node ke 2 parts hote hain:

data

next (pointer)

📌 Types of Insertion

1️⃣ Insert at Beginning

2️⃣ Insert at End

3️⃣ Insert at Position (Middle)

🔹 1. Insert at Beginning

💡 Logic

New node banao

New node ka next = head

Head ko new node bana do

📜 Code

```
public void insertAtBeginning(int data){
    Node newNode = new Node(data);
    newNode.next = head;
    head = newNode;
}
```

🧪 Dry Run

Initial:

10 -> 20 -> 30 -> null
head = 10

Insert: 5

Step:

newNode = 5
newNode.next = 10
head = 5

Final:

5 -> 10 -> 20 -> 30 -> null

🔹 2. Insert at End

💡 Logic

New node banao

Last node tak jao

Last node ka next = newNode

📜 Code

```
public void insertAtEnd(int data){
    Node newNode = new Node(data);

    if(head == null){
        head = newNode;
        return;
    }

    Node temp = head;
    while(temp.next != null){
        temp = temp.next;
    }

    temp.next = newNode;
}
```

🧪 Dry Run

Initial:

10 -> 20 -> 30 -> null

Insert: 40

Step:

temp moves: 10 -> 20 -> 30
30.next = 40

Final:

10 -> 20 -> 30 -> 40 -> null

🔹 3. Insert at Position (Middle)

💡 Logic

Position tak traverse karo

New node insert karo

Links update karo

📜 Code

```
public void insertAtPosition(int data, int position){
    Node newNode = new Node(data);

    if(position == 1){
        newNode.next = head;
        head = newNode;
        return;
    }

    Node temp = head;

    for(int i = 1; i < position - 1; i++){
        temp = temp.next;
    }

    newNode.next = temp.next;
    temp.next = newNode;
}
```

🧪 Dry Run

Initial:

10 -> 20 -> 30 -> null

Insert: 25 at position 3

Step:

temp = 20
newNode.next = 30
20.next = 25

Final:

10 -> 20 -> 25 -> 30 -> null

🧩 Full Final Code (Interview Ready)

```java
package Day_23;

public class LinkedListInsert {

    class Node {
        int data;
        Node next;

        Node(int data){
            this.data = data;
            this.next = null;
        }
    }

    Node head;

    // Insert at Beginning
    public void insertAtBeginning(int data){
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    // Insert at End
    public void insertAtEnd(int data){
        Node newNode = new Node(data);

        if(head == null){
            head = newNode;
            return;
        }

        Node temp = head;
        while(temp.next != null){
            temp = temp.next;
        }

        temp.next = newNode;
    }

    // Insert at Position
    public void insertAtPosition(int data, int position){
        Node newNode = new Node(data);

        if(position == 1){
            newNode.next = head;
            head = newNode;
            return;
        }

        Node temp = head;

        for(int i = 1; i < position - 1; i++){
            temp = temp.next;
        }

        newNode.next = temp.next;
        temp.next = newNode;
    }

    // Display
    public void display(){
        Node temp = head;
        while(temp != null){
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }
```

```
        System.out.println("null");
    }

    public static void main(String[] args) {
        LinkedListInsert list = new LinkedListInsert();

        System.out.println("The Give List InsertAt Beginning Is:");
        list.insertAtBeginning(10);
        list.display();

        System.out.println("The Give List InsertAt End Is:");
        list.insertAtEnd(20);
        list.insertAtEnd(30);
        list.display();

        System.out.println("The Give List InsertAt Position Is:");
        list.insertAtPosition(25, 3);
        list.display();
    }
}
```

⚠️ Important Edge Cases

✓ Empty list

✓ Position = 1

✓ Position > length

✓ Single node list